Operating System Lab Assignment-2

Implementation of Banker's Algorithm for Deadlock Avoidance

Student Details

Name: Payal
Course: BSC Data Science
Subject: Operating System
Course Code: ETCCOS204

Aim

To implement Banker's Algorithm to check whether the system is in a safe state and determine the safe sequence.

Introduction

Deadlock is a condition where processes cannot proceed because they are waiting for resources held by others. Banker's Algorithm avoids deadlock by ensuring the system stays in a safe state.

Objectives

- Understand resource allocation
- Calculate Need Matrix
- Implement Banker's Algorithm
- Determine safe sequence
- Analyze system safety

Algorithm

1. Input data
2. Calculate Need = Max - Allocation
3. Apply safety algorithm
4. Find safe sequence

Python Code

```
def bankers_algorithm(processes, resources, allocation, maximum, available):
    need = [[maximum[i][j] - allocation[i][j] for j in range(resources)] for i in
```

```
range(processes)]
    finish = [False] * processes
    safe_sequence = []
    work = available.copy()

    while len(safe_sequence) < processes:
        found = False
        for i in range(processes):
            if not finish[i]:
                if all(need[i][j] <= work[j] for j in range(resources)):
                    for j in range(resources):
                        work[j] += allocation[i][j]
                    safe_sequence.append(i)
                    finish[i] = True
                    found = True
        if not found:
            return "System is not in safe state"
    return safe_sequence
```

Output

Safe Sequence: [1, 3, 4, 0, 2]

Conclusion

Banker's Algorithm helps avoid deadlock by ensuring safe execution sequence.